

Sprint 2: Tính năng Ghi chú thông thường (Public Notes)

Thẻ 1: [1h] Chốt cấu trúc dữ liệu Note & Danh sách API

 **Người phụ trách:** Cả team (Họp ngắn)

 **Mô tả:**

Trước khi BE viết API và FE làm form, cả nhóm phải thống nhất một "cái Note" gồm những trường dữ liệu (fields) nào để không bị lệch pha khi ghép code.

Checklist:

- ☐ **Chốt dữ liệu:** Thống nhất Object của 1 Ghi chú gồm: id (chuỗi ngẫu nhiên), title (tiêu đề), content (nội dung), createdAt (ngày tạo), updatedAt (ngày sửa cuối).
- ☐ **Chốt URL:** Các API sẽ có dạng: /api/notes/:topic (ví dụ: /api/notes/hoc-tap).
- ☐ **Chốt tham số:** BE xác nhận sẽ nhận dữ liệu JSON từ FE dưới dạng { "title": "...", "content": "..." }.

Thẻ 2: [4h] (BE) API Ghi chú - Lấy danh sách & Thêm mới (GET, POST)

 **Người phụ trách:** Backend Dev

 **Mô tả:**

Viết 2 API đầu tiên để khởi tạo file JSON theo chủ đề, lấy danh sách ghi chú và thêm ghi chú mới. Dữ liệu lưu tại data/notes/[tên-chủ-đề].json.

Checklist:

- ☐ Tạo sẵn thư mục con data/notes/ trong source code.
- ☐ **API GET /api/notes/:topic:** Nhận topic từ URL. Dùng fs.readFileSync đọc file tương ứng (VD: hoc-tap.json). Nếu file chưa tồn tại (bị lỗi báo không thấy file), tự động trả về mảng rỗng [].
- ☐ **API POST /api/notes/:topic:** Nhận title, content từ FE. Tự động sinh id (dùng Date.now().toString()) và sinh ngày giờ hiện tại (createdAt).
- ☐ Đẩy Note mới vào mảng dữ liệu hiện tại, dùng fs.writeFileSync lưu đè lại vào file.
- ☐ Dùng Postman test tạo thử 2 ghi chú, chụp ảnh kết quả quảng vào thẻ này.

Thẻ 3: [3h] (BE) API Ghi chú - Sửa & Xóa (PUT, DELETE)

 **Người phụ trách:** Backend Dev

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

Mô tả:

Viết 2 API còn lại để chỉnh sửa nội dung hoặc xóa hẳn một ghi chú khỏi hệ thống dựa vào ID của ghi chú đó.

Checklist:

- [] **API PUT** /api/notes/:topic/:id: Nhận title, content mới. Đọc file JSON -> Duyệt mảng tìm Note có id trùng khớp -> Cập nhật nội dung & đổi updatedAt -> Lưu lại file.
- [] **API DELETE** /api/notes/:topic/:id: Đọc file JSON -> Dùng hàm array.filter() để lọc bỏ Note có id trùng khớp -> Lưu mảng mới (đã xóa phần tử) lại vào file.
- [] Test bằng Postman (Sửa 1 cái, Xóa 1 cái). Báo cho FE biết các API Note đã sẵn sàng.

Thẻ 4: [5h] (FE) UI Danh sách & Thêm mới Ghi chú

 **Người phụ trách:** Frontend Dev

Mô tả:

Tạo màn hình chính của ứng dụng: Chọn chủ đề, xem danh sách các thẻ ghi chú và Form để tạo ghi chú mới.

Checklist:

- [] **Layout Danh sách:** Tạo component hiển thị Menu chọn Chủ đề (VD: nút Tabs: Học tập, Công việc, Cá nhân). Tạo Grid hiển thị các Ghi chú dạng Thẻ (Cards).
- [] **Ghép API GET:** Dùng useEffect gọi API GET /api/notes/:topic mỗi khi người dùng đổi chủ đề -> Đổ dữ liệu ra các thẻ Note.
- [] **Form Thêm mới:** Tạo Modal hoặc Form nằm phía trên danh sách (Gồm Input Tiêu đề, Textarea Nội dung, nút Lưu).
- [] **Ghép API POST:** Khi bấm Lưu -> Gọi API POST /api/notes/:topic.
- [] Báo thành công -> Gọi lại API GET để tự động làm mới danh sách (không cần F5) -> Reset (xóa trắng) Form.

Thẻ 5: [4h] (FE) UI Cập nhật & Xóa Ghi chú

 **Người phụ trách:** Frontend Dev

Mô tả:

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

Thêm các nút thao tác lên từng Thẻ Ghi chú (Note Card) để người dùng có thể Sửa hoặc Xóa, sau đó ghép với API của BE.

✅ Checklist:

- [] **Tính năng Xóa:** Thêm nút "Xóa" (icon thùng rác) trên mỗi thẻ Note. Bấm vào hiện confirm('Bạn có chắc muốn xóa không?').
- [] **Ghép API DELETE:** Nếu User bấm OK -> Gọi API DELETE /api/notes/:topic/:id -> Báo thành công -> Gọi lại API GET để làm mới danh sách.
- [] **Tính năng Sửa:** Thêm nút "Sửa" (icon cây bút). Bấm vào -> Đổ dữ liệu cũ (title, content) của note đó ra form.
- [] **Ghép API PUT:** Người dùng sửa xong bấm Lưu -> Gọi API PUT /api/notes/:topic/:id -> Làm mới danh sách.

Thẻ 6: [2h] (PM) Nghiệm thu tính năng (Testing Sprint 2)

👤 **Người phụ trách:** PM

📄 Mô tả:

PM sẽ kiểm tra luồng từ đầu đến cuối của nhóm tính năng quản lý ghi chú. Chú ý test các trường hợp người dùng nhập thiếu dữ liệu hoặc chuyển đổi qua lại giữa các chủ đề.

✅ Checklist:

- [] Mở tab "Học tập": Thêm 2 ghi chú mới -> Chuyển sang tab "Công việc" (Danh sách phải trống) -> Chuyển lại tab "Học tập" (2 ghi chú vẫn phải còn).
- [] Bấm Sửa 1 ghi chú -> Đổi tiêu đề -> F5 trình duyệt xem tiêu đề có giữ nguyên nội dung mới sửa không.
- [] Bấm Xóa 1 ghi chú -> F5 trình duyệt xem ghi chú đó có thực sự bị xóa mất không.
- [] Mở thư mục code của Backend: Vào data/notes/, mở file JSON bằng trình soạn thảo code để xem cấu trúc mảng bên trong có đúng chuẩn không.
- [] Phát hiện lỗi thì comment tag @FE hoặc @BE. Nếu mọi thứ mượt mà -> Kéo thẻ sang Done!
- [] **Trước khi Dev làm xong:** PM phải vào file Google Sheets viết sẵn các kịch bản Testcase (Khoảng 5-7 dòng cho Note thường, 5-7 dòng cho Note kín).

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

- [] **Khi Dev làm xong:** PM tiến hành thao tác trên web đúng theo các bước trong file Testcase.
- [] Cập nhật trạng thái Pass/Fail vào file Sheets.
- [] **Quy trình báo lỗi:** Nếu có thẻ nào Fail -> Bắt buộc phải tạo Card mới trên Trello (Cột Bugs), ghi rõ *Các bước tái hiện*, *Kết quả mong đợi*, *Kết quả thực tế* và tag tên Dev vào sửa. Không báo lỗi qua tin nhắn rác.

Hướng dẫn thực hiện Sprint 2: Tính năng Ghi chú thông thường (Public Notes).

PHẦN 1: THỰC HIỆN BACKEND (Cập nhật file server.js)

Trong thư mục backend, Backend Dev mở file server.js đã làm ở Sprint 1. Thêm khối mã dưới đây vào ngay trước dòng app.listen. Hãy chú ý các phần **ghi chú (Comments)** được đóng khung cẩn thận để team hiểu rõ chức năng và tránh sửa nhầm.

```
/**
 * =====
 * MODULE: QUẢN LÝ GHI CHÚ THÔNG THƯỜNG (PUBLIC NOTES)
 * Author: [Điền tên Backend Dev]
 * Date: [Ngày thực hiện]
 * Description: Nhóm API hỗ trợ CRUD cho ghi chú theo chủ đề.
 *
 * [CẢNH BÁO TRÁNH XUNG ĐỘT]:
 * - FE Team: Các API này nhận và trả về dữ liệu chuẩn JSON. Không tự ý đổi tên key.
 * - BE Team: Nếu đổi đường dẫn lưu file (notesDir), phải báo cáo với PM.
 * =====
 */

const notesDir = path.join(__dirname, 'data', 'notes');
// Khởi tạo thư mục tự động nếu chưa tồn tại
if (!fs.existsSync(notesDir)) {
  fs.mkdirSync(notesDir, { recursive: true });
}

const getFilePath = (topic) => path.join(notesDir, `${topic}.json`);
// 1. Lấy danh sách ghi chú (GET)
app.get('/api/notes/:topic', (req, res) => {
  const filePath = getFilePath(req.params.topic);
  try {
    if (!fs.existsSync(filePath)) return res.json([]);
    const data = fs.readFileSync(filePath, 'utf8');
    res.json(JSON.parse(data));
  } catch (error) {
    res.status(500).json({ message: "Lỗi đọc danh sách ghi chú" });
  }
});
// 2. Thêm mới ghi chú (POST)
app.post('/api/notes/:topic', (req, res) => {
  const filePath = getFilePath(req.params.topic);
  try {
    let notes = fs.existsSync(filePath) ? JSON.parse(fs.readFileSync(filePath, 'utf8')) : [];

```

```
const newNote = {
  id: Date.now().toString(),
  title: req.body.title || "Không tiêu đề",
  content: req.body.content || "",
  createdAt: new Date().toISOString(),
  updatedAt: new Date().toISOString()
};

notes.push(newNote);
fs.writeFileSync(filePath, JSON.stringify(notes, null, 2), 'utf8');
res.json({ success: true, note: newNote });
} catch (error) {
  res.status(500).json({ message: "Lỗi thêm ghi chú" });
}
});

// 3. Sửa ghi chú (PUT)
app.put('/api/notes/:topic/:id', (req, res) => {
  const filePath = getFilePath(req.params.topic);
  try {
    let notes = JSON.parse(fs.readFileSync(filePath, 'utf8'));
    const index = notes.findIndex(n => n.id === req.params.id);

    if (index !== -1) {
      notes[index].title = req.body.title;
      notes[index].content = req.body.content;
      notes[index].updatedAt = new Date().toISOString();

      fs.writeFileSync(filePath, JSON.stringify(notes, null, 2), 'utf8');
      return res.json({ success: true, message: "Đã sửa thành công" });
    }
    res.status(404).json({ message: "Không tìm thấy ghi chú" });
  } catch (error) {
    res.status(500).json({ message: "Lỗi cập nhật ghi chú" });
  }
});

// 4. Xóa ghi chú (DELETE)
app.delete('/api/notes/:topic/:id', (req, res) => {
  const filePath = getFilePath(req.params.topic);
  try {
    let notes = JSON.parse(fs.readFileSync(filePath, 'utf8'));
    const newNotes = notes.filter(n => n.id !== req.params.id);
    fs.writeFileSync(filePath, JSON.stringify(newNotes, null, 2), 'utf8');
    res.json({ success: true, message: "Đã xóa thành công" });
  }
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
    } catch (error) {
      res.status(500).json({ message: "Lỗi xóa ghi chú" });
    }
  });
```

PHẦN 2: THỰC HIỆN FRONTEND (ReactJS)

Trong thư mục frontend/src, tạo file Notes.jsx.

Mã nguồn dưới đây đã được quy hoạch thành **3 vùng rõ ràng (State, Logic, UI)** thông qua các khối ghi chú. Điều này giúp các FE Dev khác trong nhóm (nếu có) biết chính xác nên tìm và sửa code ở đâu mà không đụng chạm vào phần của người khác.

```
/**
 * =====
 * COMPONENT: GIAO DIỆN QUẢN LÝ GHI CHÚ (Notes.jsx)
 * Author: [Điền tên Frontend Dev]
 *
 * [CẢNH BÁO TRÁNH XUNG ĐỘT]:
 * - Vùng 1 & 2 (State & Logic): Chỉ người phụ trách tích hợp API mới được sửa.
 * - Vùng 3 (Render UI): Các bạn phụ trách CSS/HTML có thể tùy chỉnh ở đây.
 * =====
 */
import React, { useState, useEffect } from 'react';
function Notes() {
  /* =====
   VÙNG 1: KHỞI TẠO STATE (Trạng thái dữ liệu)
   ===== */
  const [topic, setTopic] = useState('hoc-tap');
  const [notes, setNotes] = useState([]);
  const [formData, setFormData] = useState({ id: null, title: '', content: '' });
  /* =====
   VÙNG 2: XỬ LÝ LOGIC & GỌI API (Fetch, Save, Delete)
   ===== */
  const fetchNotes = () => {
    fetch(`http://localhost:5000/api/notes/${topic}`)
      .then(res => res.json())
      .then(data => setNotes(data));
  };
  useEffect(() => { fetchNotes(); }, [topic]);
  const handleSave = () => {
    const method = formData.id ? 'PUT' : 'POST';
    const url = formData.id
      ? `http://localhost:5000/api/notes/${topic}/${formData.id}`
      : `http://localhost:5000/api/notes/${topic}`;
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
    fetch(url, {
      method: method,
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: formData.title, content: formData.content })
    })
    .then(res => res.json())
    .then(() => {
      fetchNotes();
      setFormData({ id: null, title: '', content: '' });
    });
  });

  const handleDelete = (id) => {
    if(window.confirm('Bạn có chắc muốn xóa ghi chú này?')) {
      fetch(`http://localhost:5000/api/notes/${topic}/${id}`, { method: 'DELETE' })
        .then(() => fetchNotes());
    }
  };

  const handleEdit = (note) => setFormData({ id: note.id, title: note.title, content:
note.content });

  /* =====
  VÙNG 3: RENDER GIAO DIỆN (UI/CSS)
  ===== */
  return (
    <div style={{ padding: '20px' }}>
      <h2>Ghi chú Công khai</h2>

      /* 3.1. Vùng chọn chủ đề */
      <div style={{ marginBottom: '20px' }}>
        <strong>Chủ đề: </strong>
        <select value={topic} onChange={(e) => setTopic(e.target.value)}>
          <option value="hoc-tap">Học tập</option>
          <option value="cong-viec">Công việc</option>
          <option value="ca-nhan">Cá nhân</option>
        </select>
      </div>

      /* 3.2. Form Nhập liệu */
      <div style={{ border: '1px solid #ccc', padding: '10px', marginBottom: '20px'
}}>

        <h3>{formData.id ? 'Sửa ghi chú' : 'Thêm ghi chú mới'}</h3>
        <input
          placeholder="Tiêu đề" value={formData.title}
          onChange={e => setFormData({...formData, title: e.target.value})}
          style={{ display: 'block', width: '100%', marginBottom: '10px' }}
        />
      </div>
    </div>
  );
}
```

```
<textarea
  placeholder="Nội dung" value={formData.content}
  onChange={e => setFormData({...formData, content: e.target.value})}
  style={{ display: 'block', width: '100%', height: '80px', marginBottom:
'10px' }}
/>
<button onClick={handleSave}>{formData.id ? 'Cập nhật' : 'Thêm mới'}</button>
{formData.id && (
  <button onClick={() => setFormData({ id: null, title: '', content: '' })}
style={{marginLeft: '10px'}}>Hủy</button>
)}
</div>
{/* 3.3. Danh sách thẻ ghi chú */}
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px' }}>
  {notes.length === 0 && <p>Chưa có ghi chú nào.</p>}
  {notes.map(note => (
    <div key={note.id} style={{ border: '1px solid #007bff', padding: '15px',
borderRadius: '5px' }}>
      <h4 style={{ margin: '0 0 10px 0' }}>{note.title}</h4>
      <p style={{ whiteSpace: 'pre-wrap' }}>{note.content}</p>
      <div style={{ marginTop: '10px' }}>
        <button onClick={() => handleEdit(note)} style={{ marginRight: '10px'
}}>Sửa</button>
        <button onClick={() => handleDelete(note.id)} style={{ color: 'red'
}}>Xóa</button>
      </div>
    </div>
  ))}
</div>
</div>
);
}
export default Notes;
```

PHẦN 3: LƯU TRỮ VÀ ĐẨY CODE LÊN GITHUB (Quy trình chống Conflict)

Để tránh tối đa tình trạng xung đột mã nguồn (Merge Conflict) khi làm việc nhóm, trước khi đẩy code của mình lên, bạn **bắt buộc phải lấy code mới nhất của người khác về máy** để kiểm tra.

Mở Terminal tại thư mục gốc của dự án (do-an-ghi-chu) và thực hiện theo đúng thứ tự sau:

1. Lấy code mới nhất từ team về (Pull):

```
git pull origin main
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

(Nếu không có báo lỗi đỏ nào, bạn có thể tự tin làm các bước tiếp theo. Nếu có lỗi CONFLICT, cần báo ngay cho PM để cùng xử lý).

2. Ghi nhận các file bạn vừa thay đổi:

```
git add .
```

3. Tạo Commit với mô tả rõ ràng:

```
git commit -m "feat(Notes): Hoàn thành API CRUD và Giao diện Ghi chú (Sprint 2)"
```

(Mẹo: Ghi chú "feat(Notes)" giúp nhóm biết nhanh commit này liên quan đến tính năng Note).

4. Đẩy code lên nhánh chung (Push):

```
git push origin main
```